

Cyber-Toolbox

Documentação do Código-Fonte

Martinho Caeiro (23917)
Mestrado de Engenharia em Segurança Informática
Linguagens de Programação Dinâmicas

2 de fevereiro de 2026

Conteúdo

1	Introdução	2
2	Menu Principal	2
2.1	menu_launcher	2
2.2	scripts_config	3
3	Scripts Individuais	4
3.1	log_analyzer	4
3.2	messenger	6
3.3	password_manager	8
3.4	port_knocker	10
3.5	port_scanner	11
3.6	syn_flooder	12
3.7	udp_flooder	13

1 Introdução

Este documento contém a documentação técnica extraída do código-fonte do projeto Cyber-Toolbox. Todos os scripts foram desenvolvidos em Python 3 com comentários em inglês e mensagens de saída em português (pt-PT).

2 Menu Principal

2.1 menu_launcher

Ficheiro: `src/menu_launcher.py`

Autor: Martinho Caeiro (23917)

Descrição:

Simple menu launcher to manage/run scripts in the `./scripts` folder.

Uso:

```
python3 menu_launcher.py
```

Exemplo:

```
python3 menu_launcher.py
```

Secções do código:

- Helpers
- Script execution
- Main loop

Funções e Classes principais:

- `ensure_dirs()`
- `clear_screen()`
- `show_menu()`
- `prompt_args(arg_defs)`
- `run_script(path, args)`
- `main_menu()`

2.2 scripts_config

Ficheiro: `src/scripts_config.py`

Autor: Martinho Caeiro (23917)

Descrição:

Defines the configuration of available scripts, including names, files, and required arguments.

3 Scripts Individuais

3.1 log_analyzer

Ficheiro: `src/scripts/log_analyzer.py`

Autor: Martinho Caeiro (23917)

Descrição:

Analyze Apache/Nginx access logs and OpenSSH auth logs.

Funcionalidades:

- Parse Apache/Nginx combined log lines and OpenSSH auth.log lines.
- Extract IP, timestamp, service, status (success/fail), and details.
- Optionally resolve country via GeoLite2 DB (pass `-geoip-db`).
- Emit JSON and CSV reports grouped per event.

Uso:

```
python log_analyzer.py --files access.log auth.log --outdir reports --geoip-db /path/GeoLite2
```

Exemplo:

```
python log_analyzer.py --files access.log auth.log --outdir reports
```

Secções do código:

- Dependencies
- Patterns
- Resolvers
- Parsers
- Reporting
- CLI

Funções e Classes principais:

- Classe: `GeoIPResolver`
- `parse_apache_line(line)`
- `parse_ssh_line(line)`
- `parse_ufw_line(line)`
- `analyze_files(file_paths, geoip_db, outdir)`
- `find_default_geoip_db()`
Look for GeoLite2 DB files inside the repository 'src/data' folder.

- `summarize_events(events)`

Print a small summary: counts per service and top countries.

- `main(argv)`

- `country_for(self, ip)`

3.2 messenger

Ficheiro: `src/scripts/messenger.py`

Autor: Martinho Caeiro (23917)

Descrição:

Simple secure messenger (server + client CLI) for the Cyber-Toolbox project.

Funcionalidades:

- Server generates an RSA keypair (`data/keys/`) with `init-keys`.
- Clients connect to the server (TCP) and send JSON commands.
- Messages are stored on the server only, in `data/messages/`.
- Each message is hybrid-encrypted: a Fernet payload + the Fernet key encrypted with the server RSA public key.
- Clients can list/download/delete messages where they are participants.
- Server can produce backups (symmetric password-protected or asymmetric).

Uso:

```
python messenger.py init-keys
```

```
python messenger.py serve --host 127.0.0.1 --port 9009
```

```
python messenger.py send --from alice --to bob --message "Olá"
```

Exemplo:

```
python messenger.py list --user alice
```

Nota:

This script is intentionally simple and synchronous. It is meant for local testing and demonstration, not production use.

Secções do código:

- Dependencies
- Key management
- Message storage
- Server
- Client & CLI

Funções e Classes principais:

- `ensure_dirs()`
- `generate_rsa_keys(key_size)`

- `load_private_key()`
- `load_public_key()`
- `hybrid_encrypt(plaintext)`
Encrypt plaintext with a fresh Fernet key, then encrypt that key with RSA public key.
- `hybrid_decrypt(b64_enc_sym_key, b64_payload)`
- `store_message(sender, recipients, text)`
- `list_messages_for(user)`
- `load_message_file(mid)`
- `delete_message(mid, actor)`
- `backup_all_sym(password)`
Create a password-protected backup file (Fernet with key derived from password).
- `backup_all_asym()`
Create an asymmetric backup encrypted with server public key.
- Classe: `ThreadedTCPHandler`
- `run_server(host, port)`
- `client_send(host, port, payload)`
- `print_response(resp)`
Print a response payload in pt-PT.
- `main()`
- `handle(self)`

3.3 password_manager

Ficheiro: `src/scripts/password_manager.py`

Autor: Martinho Caeiro (23917)

Descrição:

Simple password manager with RSA + Fernet encryption and TOTP 2FA.

Uso:

```
python3 password_manager.py --init
python3 password_manager.py --create --url example.com --user alice --pass s3cr3t
python3 password_manager.py --list --otp 123456
python3 password_manager.py --view <id> --otp 123456
python3 password_manager.py --update <id> [--url ..] [--user ..] [--pass ..] --otp 123456
python3 password_manager.py --delete <id> --otp 123456
```

Exemplo:

```
python3 password_manager.py --list --otp 123456
```

Files created (in the same folder):

- `private_key.pem` (PEM, kept secret)
- `public_key.pem` (PEM)
- `totp_secret.txt` (base32 secret for TOTP)
- `records.json` (encrypted records database)

Nota:

This is a simple educational implementation. Do not use for production secrets.

Secções do código:

- Dependencies
- Paths
- Key management
- Commands
- CLI

Funções e Classes principais:

- `ensure_files_dir()`
- `init_keys_and_totp()`
Generate RSA keypair and a new TOTP secret.
- `load_public_key()`
- `load_private_key()`
- `load_records()`

- `save_records(recs)`
- `encrypt_record_plaintext(plaintext_bytes)`
 Encrypt plaintext bytes using Fernet and encrypt the key with RSA public key.
- `decrypt_record(enc_cipher_b64, enc_key_b64)`
- `require_otp(code)`
- `cmd_create(args)`
- `cmd_list(args)`
- `cmd_view(args)`
- `cmd_delete(args)`
- `cmd_update(args)`
- `parse_args()`
- `main()`
- `run_interactive_menu()`
- Classe: A
- Classe: A
- Classe: A
- Classe: A
- Classe: A

3.4 port_knocker

Ficheiro: `src/scripts/port_knocker.py`

Autor: Martinho Caeiro (23917)

Descrição:

Sends a sequence of packets to specific ports of a remote server to enable SSH access.

Uso:

```
python3 port_knocker.py <host> <port1> <port2> <port3>
```

Exemplo:

```
python3 port_knocker.py 192.168.1.100 7000 8000 9000
```

Secções do código:

- Port knocking
- Main loop

Funções e Classes principais:

- `send_knock(host, port)`
- `main()`

3.5 port_scanner

Ficheiro: `src/scripts/port_scanner.py`

Autor: Martinho Caeiro (23917)

Descrição:

Scans ports on multiple targets to identify which are open.

Uso:

```
python3 port_scanner.py <target1,target2,...> <start_port> <end_port>
```

Exemplo:

```
python3 port_scanner.py target1.com,target2.com,127.0.0.1 1 1024
```

Secções do código:

- Scanner
- CLI

Funções e Classes principais:

- `scan_host(host, start_port, end_port, timeout)`
Scan ports on a specific host.
- `main()`

3.6 syn_flooder

Ficheiro: `src/scripts/syn_flooder.py`

Autor: Martinho Caeiro (23917)

Descrição:

Sends TCP SYN packets to simulate a denial-of-service attack (SYN Flood/DoS). Uses Scapy for low-level packet crafting.

Uso:

```
python3 syn_flooder.py [--target HOST] [--port PORT] [--duration SECONDS] [--threads N]
```

Exemplo:

```
python3 syn_flooder.py --target 192.168.1.1 --port 80 --duration 10
```

Secções do código:

- SYN Flooder functions

Funções e Classes principais:

- `generate_random_ip()`
Generate a random spoofed source IP address.
- `generate_random_port()`
Generate a random source port.
- `syn_flood_worker(target, target_port, duration, packets_sent, lock)`
Worker thread that sends TCP SYN packets to a target.
- `flood_target(target, target_port, duration, num_threads)`
Send SYN flood with multiple threads.
- `main()`

3.7 udp_flooder

Ficheiro: `src/scripts/udp_flooder.py`

Autor: Martinho Caeiro (23917)

Descrição:

Sends UDP packets to a target (UDP Flood attack).

Uso:

```
python3 udp_flooder.py [--target HOST] [--port PORT] [--duration SECONDS] [--threads N]
```

Exemplo:

```
python3 udp_flooder.py --target 192.168.1.1 --port 53 --duration 10
```

Secções do código:

- Workers
- Flood logic
- CLI

Funções e Classes principais:

- `udp_flood_worker(target, port, packet_size, duration, packets_sent)`
Worker thread that sends UDP packets to a target.
- `flood_target(target, port, packet_size, duration, num_threads)`
Send UDP flood with multiple threads.
- `main()`